

# part G. Interfaces

I mentioned back in that although type aliases for `{ ... }` object types are a way to describe object shapes, TypeScript also includes an “interface” feature many developers prefer. Interfaces are another way to declare an object shape with an associated name. Interfaces are in many ways similar to aliased object types but are generally preferred for their more readable error messages, speedier compiler performance, and better interoperability with classes.

## Type Aliases Versus Interfaces

Here is a quick recap of the syntax for how an aliased object type would describe an object with a `born: number` and `name: string`:

```
type Poet = {  
  born: number;  
  name: string;  
};
```

Here is the equivalent syntax for an interface:

```
interface Poet {  
  born: number;  
  name: string;  
}
```

The two syntaxes are almost identical.

TypeScript’s assignability checking and error messages for interfaces also work and look just about the same as they do for object types. The following assignability errors for assigning to the `valueLater` variable would be roughly the same if `Poet` was an interface or type alias:

```
let valueLater: Poet;  
  
// Ok  
valueLater = {
```

```
    born: 1935,
    name: 'Sara Teasdale',
  };

valueLater = "Emily Dickinson";
// Error: Type 'string' is not assignable to 'Poet'.

valueLater = {
  born: true,
  // Error: Type 'boolean' is not assignable to type 'number'.
  name: 'Sappho'
};
```

However, there are a few key differences between interfaces and type aliases:

- As you’ll see later in this part, interfaces can “merge” together to be augmented—a feature particularly useful when working with third-party code such as built-in globals or npm packages.
- As you’ll see in the next part, , interfaces can be used to type check the structure of class declarations while type aliases cannot.
- Interfaces are generally speedier for the TypeScript type checker to work with: they declare a named type that can be cached more easily internally, rather than a dynamic copy-and-paste of a new object literal the way type aliases do.
- Because interfaces are considered named objects rather than an alias for an unnamed object literal, their error messages are more likely to be readable in hard edge cases.

For the latter two reasons and to maintain consistency, the rest of this material and its associated projects default to using interfaces over aliased object shapes. I generally recommend using interfaces whenever possible (i.e., until you need features such as union types from type aliases).

## Types of Properties

JavaScript objects can be wild and wacky in real-world usage, including getters and setters, properties that only sometimes exist, or accepting any arbitrary property names.

TypeScript provides a set of type system tools for interfaces to help us model that wackiness.

Because interfaces and type aliases behave so similarly, the following types of properties introduced in this part are all also usable with aliased object types.

## Optional Properties

As with object types, interface properties don't all have to be required in the object. You can indicate an interface's property is optional by including a `?` before the `:` in its type annotation.

This `Part` interface requires only a `pages` property and optionally allows an `author`. Objects adhering to it may provide `author` or leave it out as long as they provide `pages`:

```
interface Part {  
  author?: string;  
  pages: number;  
};  
  
// Ok  
const ok: Part = {  
  author: "Rita Dove",  
  pages: 80,  
};  
  
const missing: Part = {  
  author: "Rita Dove"  
};  
// Error: Property 'pages' is missing in type  
// '{ author: string; }' but required in type 'Part'.
```

The same caveats around the difference between optional properties and properties whose type happens to include `undefined` in a type union apply to interfaces as well as object types. will describe TypeScript's strictness settings around optional properties.

## Read-Only Properties

You may sometimes wish to block users of your interface from reassigning properties of objects adhering to that interface. TypeScript allows you to add a `readonly` modifier be-

fore a property name to indicate that once set, that property should not be set to a different value. These `readonly` properties can be read from normally, but not reassigned to anything new.

For example, the `text` property in the below `Page` interface gives back a `string` when accessed, but causes a type error if assigned a new value:

```
interface Page {
  readonly text: string;
}

function read(page: Page) {
  // Ok: reading the text property doesn't attempt to modify it
  console.log(page.text);

  page.text += "!";
  //    ~~~~
  // Error: Cannot assign to 'text'
  // because it is a read-only property.
}
```

Note that `readonly` modifiers exist only in the type system, and only apply to the usage of that interface. It won't apply to an object unless that object is used in a location that declares it to be of that interface.

In this continuation of the `read` example, the `text` property is allowed to be modified outside of the `read` function because its parent object isn't explicitly described as a `Text` until inside the function. `pageIsh` is allowed to be used as a `Page` because a writable property is assignable to a `readonly` property (mutable properties can be read from, which is all a `readonly` property needs):

```
const pageIsh = {
  text: "Hello, world!",
};

// Ok: pageIsh is an inferred object type with text, not a Page
page.text += "!";

// Ok: read takes in Page, which happens to
// be a more specific version of pageIsh's type
read(pageIsh);
```

Declaring the variable `pageIsh` with the explicit type annotation `: Page` would have indicated to TypeScript that its `text` property was `readonly`. Its inferred type, however, was not `readonly`.

Read-only interface members are a handy way to make sure areas of code don't unexpectedly modify objects they're not meant to. However, remember that they're a type system construct only and don't exist in the compiled JavaScript output code. They only protect from modification during development with the TypeScript type checker.

## Functions and Methods

It's very common in JavaScript for object members to be functions. TypeScript therefore allows declaring interface members as being the function types previously covered in .

TypeScript provides two ways of declaring interface members as functions:

- *Method* syntax: declaring that a member of the interface is a function intended to be called as a member of the object, like `member(): void`
- *Property* syntax: declaring that a member of the interface is equal to a standalone function, like `member: () => void`

The two declaration forms are an analog for the two ways you can declare a JavaScript object as having a function.

Both `method` and `property` members shown here are functions that may be called with no parameters and return a `string`:

```
interface HasBothFunctionTypes {
  property: () => string;
  method(): string;
}

const hasBoth: HasBothFunctionTypes = {
  property: () => "",
  method() {
    return "";
  }
};
```

```
hasBoth.property(); // Ok
hasBoth.method(); // Ok
```

Both forms can receive the `?` optional modifier to indicate they don't need to be provided:

```
interface OptionalReadOnlyFunctions {
  optionalProperty?: () => string;
  optionalMethod?(): string;
}
```

Method and property declarations can mostly be used interchangeably. The main differences between them that I'll cover in this material are:

- Methods cannot be declared as `readonly`; properties can.
- Interface merging (covered later in this part) treats them differently.
- Some of the operations performed on types covered in treat them differently.

Future versions of TypeScript may add the option to be more strict about the differences between methods and property functions.

For now, the general style guide I recommend is:

- Use a method function if you know the underlying function may refer to `this`, most commonly for instances of classes (covered in ).
- Use a property function otherwise.

Don't sweat it if you mix up these two, or don't understand the difference. It'll rarely impact your code unless you're being intentional about `this` scoping and which form you choose.

## Call Signatures

Interfaces and object types can declare *call signatures*, which is a type system description of how a value may be called like a function. Only values that may be called in the way the call signature declares will be assignable to the in-



terface—i.e., a function with assignable parameters and return type. A call signature looks similar to a function type, but with a `:` colon instead of an `=>` arrow.

The following `FunctionAlias` and `CallSignature` types both describe the same function parameters and return type:

```
type FunctionAlias = (input: string) => number;

interface CallSignature {
  (input: string): number;
}

// Type: (input: string) => number
const typedFunctionAlias: FunctionAlias = (input) => input.length; // Ok

// Type: (input: string) => number
const typedCallSignature: CallSignature = (input) => input.length; // Ok
```

Call signatures can be used to describe functions that additionally have some user-defined property on them.

TypeScript will recognize a property added to a function declaration as adding to that function declaration's type.

The following `keepsTrackOfCalls` function declaration is given a `count` property of type `number`, making it assignable to the `FunctionWithCount` interface. It can therefore be assigned to the `hasCallCount` argument of type `FunctionWithCount`. The function at the end of the snippet was not given a `count`:

```
interface FunctionWithCount {
  count: number;
  (): void;
}

let hasCallCount: FunctionWithCount;

function keepsTrackOfCalls() {
  keepsTrackOfCalls.count += 1;
  console.log(`I've been called ${keepsTrackOfCalls.count} times!`);
}

keepsTrackOfCalls.count = 0;

hasCallCount = keepsTrackOfCalls; // Ok
```

```
function doesNotHaveCount() {
  console.log("No idea!");
}

hasCallCount = doesNotHaveCount;
// Error: Property 'count' is missing in type
// '() => void' but required in type 'FunctionWithCount'
```

## Index Signatures

Some JavaScript projects create objects meant to store values under any arbitrary string key. For these “container” objects, declaring an interface with a field for every possible key would be impractical or impossible.

TypeScript provides a syntax called an *index signature* to indicate that an interface’s objects are allowed to take in any key and give back a certain type under that key. They’re most commonly used with string keys because JavaScript object property lookups convert keys to strings implicitly. An index signature looks like a regular property definition but with a type after the key, and array brackets surrounding them, like `{ [i: string]: ... }`.

This `WordCounts` interface is declared as allowing any string key with a number value. Objects of that type aren’t bound to receiving any particular key—as long as the value is a number:

```
interface WordCounts {
  [i: string]: number;
}

const counts: WordCounts = {};

counts.apple = 0; // Ok
counts.banana = 1; // Ok

counts.cherry = false;
// Error: Type 'boolean' is not assignable to type 'number'.
```

Index signatures are convenient for assigning values to an object but aren’t completely type safe. They indicate that an object should give back a value no matter what property is being accessed.



This `publishDates` value safely gives back `Frankenstein` as a `Date` but tricks TypeScript into thinking its `Beloved` is defined even though it's `undefined`:

```
interface DatesByName {
  [i: string]: Date;
}

const publishDates: DatesByName = {
  Frankenstein: new Date("1 January 1818"),
};

publishDates.Frankenstein; // Type: Date
console.log(publishDates.Frankenstein.toString()); // Ok

publishDates.Beloved; // Type: Date, but runtime value of undefined!
console.log(publishDates.Beloved.toString()); // Ok in the type system, but...
// Runtime error: Cannot read property 'toString'
// of undefined (reading publishDates.Beloved)
```

When possible, if you're looking to store key-value pairs and the keys aren't known ahead of time, it is generally safer to use a `Map`. Its `.get` method always returns a type with `| undefined` to indicate that the key might not exist. will discuss working with generic container classes such as `Map` and `Set`.

## Mixing properties and index signatures

Interfaces are able to include explicitly named properties and catchall `string` index signatures, with one catch: each named property's type must be assignable to its catchall index signature's type. You can think of mixing them as telling TypeScript that named properties give a more specific type, and any other property falls back to the index signature's type.

Here, `HistoricalNovels` declares that all properties are type `number`, and additionally the `Oroonoko` property must exist to begin with:

```
interface HistoricalNovels {
  Oroonoko: number;
  [i: string]: number;
}
```

```
// Ok
const novels: HistoricalNovels = {
  Outlander: 1991,
  Oroonoko: 1688,
};

const missingOroonoko: HistoricalNovels = {
  Outlander: 1991,
};
// Error: Property 'Oroonoko' is missing in type
// '{ Outlander: number; }' but required in type 'HistoricalNovels'.
```

One common type system trick with mixed properties and index signatures is to use a more specific property type literal for the named property than an index signature’s primitive. As long as the named property’s type is assignable to the index signature’s—which is true for a literal and a primitive, respectively—TypeScript will allow it.

Here, `PartStarts` declares that a property under `preface` must be `0` and all other properties have the more general `number`. That means any object adhering to `PartStarts` must have a `preface` property equal to `0`:

```
interface PartStarts {
  preface: 0;
  [i: string]: number;
}

const correctPreface: PartStarts = {
  preface: 0,
  night: 1,
  shopping: 5
};

const wrongPreface: PartStarts = {
  preface: 1,
  // Error: Type '1' is not assignable to type '0'.
};
```

## Numeric index signatures

Although JavaScript implicitly converts object property lookup keys to strings, it is sometimes desirable to only allow numbers as keys for an object. TypeScript index signatures can use a `number` type instead of `string` but with the

same catch as named properties that their types must be assignable to the catchall `string` index signature's.

The following `MoreNarrowNumbers` interface would be allowed because `string` is assignable to `string | undefined`, but `MoreNarrowStrings` would not because `string | undefined` is not assignable to `string`:

```
// Ok
interface MoreNarrowNumbers {
  [i: number]: string;
  [i: string]: string | undefined;
}

// Ok
const mixesNumbersAndStrings: MoreNarrowNumbers = {
  0: '',
  key1: '',
  key2: undefined,
}

interface MoreNarrowStrings {
  [i: number]: string | undefined;
  // Error: 'number' index type 'string | undefined'
  // is not assignable to 'string' index type 'string'.
  [i: string]: string;
}
```

## Nested Interfaces

Just like object types can be nested as properties of other object types, interface types can also have properties that are themselves interface types (or object types).

This `Novel` interface contains an `author` property that must satisfy an inline object type and a `setting` property that must satisfy the `Setting` interface:

```
interface Novel {
  author: {
    name: string;
  };
  setting: Setting;
}

interface Setting {
  place: string;
}
```

```

    year: number;
}

let myNovel: Novel;

// Ok
myNovel = {
  author: {
    name: 'Jane Austen',
  },
  setting: {
    place: 'England',
    year: 1812,
  }
};

myNovel = {
  author: {
    name: 'Emily Brontë',
  },
  setting: {
    place: 'West Yorkshire',
  },
  // Error: Property 'year' is missing in type
  // '{ place: string; }' but required in type 'Setting'.
};

```

## Interface Extensions

Sometimes you may end up with multiple interfaces that look similar to each other. One interface may contain all the same members of another interface, with a few extras added on.

TypeScript allows an interface to *extend* another interface, which declares it as copying all the members of another. An interface may be marked as extending another interface by adding the `extends` keyword after its name (the “derived” interface), followed by the name of the interface to extend (the “base” interface). Doing so indicates to TypeScript that all objects adhering to the derived interface must also have all the members of the base interface.

In the following example, the `Novella` interface extends from `Writing` and thus requires objects to have at least both `Novella`’s `pages` and `Writing`’s `title` members:

```

interface Writing {
    title: string;
}

interface Novella extends Writing {
    pages: number;
}

// Ok
let myNovella: Novella = {
    pages: 195,
    title: "Ethan Frome",
};

let missingPages: Novella = {
    // ~~~~~
    // Error: Property 'pages' is missing in type
    // '{ title: string; }' but required in type 'Novella'.
    title: "The Awakening",
}

let extraProperty: Novella = {
    // ~~~~~
    // Error: Type '{ genre: string; name: string; strategy: string; }'
    // is not assignable to type 'Novella'.
    //   Object literal may only specify known properties,
    //   and 'strategy' does not exist in type 'Novella'.
    pages: 300,
    strategy: "baseline",
    style: "Naturalism"
};

```

Interface extensions are a nifty way to represent that one type of entity in your project is a superset (it includes all the members of) another entity. They allow you to avoid having to type out the same code repeatedly across multiple interfaces to represent that relationship.

## Overridden Properties

Derived interfaces may *override*, or replace, properties from their base interface by declaring the property again with a different type. TypeScript's type checker will enforce that an overridden property must be assignable to its base property. It does so to ensure that instances of the derived interface type stay assignable to the base interface type.

Most derived interfaces that redeclare properties do so either to make those properties a more specific subset of a type union or to make the properties a type that extends from the base interface's type.

For example, this `WithNullableName` type is properly made non-nullable in `WithNonNullableName`. `WithNumericName`, however, is not allowed as `number | string` and is not assignable to `string | null`:

```
interface WithNullableName {
  name: string | null;
}

interface WithNonNullableName extends WithNullableName {
  name: string;
}

interface WithNumericName extends WithNullableName {
  name: number | string;
}
// Error: Interface 'WithNumericName' incorrectly
// extends interface 'WithNullableName'.
//   Types of property 'name' are incompatible.
//     Type 'string | number' is not assignable to type 'string | null'.
//       Type 'number' is not assignable to type 'string'.
```

## Extending Multiple Interfaces

Interfaces in TypeScript are allowed to be declared as extending multiple other interfaces. Any number of interface names separated by commas may be used after the `extends` keyword following the derived interface's name. The derived interface will receive all members from all base interfaces.

Here, the `GivesBothAndEither` has three methods: one on its own, one from `GivesNumber`, and one from `GivesString`:

```
interface GivesNumber {
  giveNumber(): number;
}

interface GivesString {
  giveString(): string;
}
```



```
interface GivesBothAndEither extends GivesNumber, GivesString {
  giveEither(): number | string;
}

function useGivesBoth(instance: GivesBothAndEither) {
  instance.giveEither(); // Type: number | string
  instance.giveNumber(); // Type: number
  instance.giveString(); // Type: string
}
```

By marking an interface as extending multiple other interfaces, you can both reduce code duplication and make it easier for object shapes to be reused across different areas of code.

## Interface Merging

One of the important features of interfaces is their ability to *merge* with each other. Interface merging means if two interfaces are declared in the same scope with the same name, they'll join into one bigger interface under that name with all declared fields.

This snippet declares a `Merged` interface with two properties: `fromFirst` and `fromSecond`:

```
interface Merged {
  fromFirst: string;
}

interface Merged {
  fromSecond: number;
}

// Equivalent to:
// interface Merged {
//   fromFirst: string;
//   fromSecond: number;
// }
```

Interface merging isn't a feature used very often in day-to-day TypeScript development. I would recommend avoiding it when possible, as it can be difficult to understand code where an interface is declared in multiple places.

However, interface merging is particularly useful for augmenting interfaces from external packages or built-in global interfaces such as `Window`. For example, when using the default TypeScript compiler options, declaring a `Window` interface in a file with a `myEnvironmentVariable` property makes a `window.myEnvironmentVariable` available:

```
interface Window {  
  myEnvironmentVariable: string;  
}  
  
window.myEnvironmentVariable; // Type: string
```

## Member Naming Conflicts

Note that merged interfaces may not declare the same name of a property multiple times with different types. If a property is already declared in an interface, a later merged interface must use the same type.

In this `MergedProperties` interface, the `same` property is allowed because it is the same in both declarations, but `different` is an error for being a different type:

```
interface MergedProperties {  
  same: (input: boolean) => string;  
  different: (input: string) => string;  
}  
  
interface MergedProperties {  
  same: (input: boolean) => string; // Ok  
  
  different: (input: number) => string;  
  // Error: Subsequent property declarations must have the same type.  
  // Property 'different' must be of type '(input: string) => string',  
  // but here has type '(input: number) => string'.  
}
```

Merged interfaces may, however, define a method with the same name and a different signature. Doing so creates a function overload for the method.

This `MergedMethods` interface creates a `different` method that has two overloads:

```
interface MergedMethods {
  different(input: string): string;
}

interface MergedMethods {
  different(input: number): string; // Ok
}
```

This part introduced how object types may be described by interfaces:

- Using interfaces instead of type aliases to declare object types
- Various interface property types: optional, read-only, function, and method
- Using index signatures for catchall object properties
- Reusing interfaces using nested interfaces and `extends` inheritance
- How interfaces with the same name can merge together

Next up will be a native JavaScript syntax for setting up multiple objects to have the same properties: classes.